onec-ai

MCP-СЕРВЕР ДЛЯ КОМАНД 1С І BAS

AI, який знає вашу конфігурацію 1С

Розробники, аналітики й підтримка отримують точні відповіді з цитатами з вашого коду — за секунди, а не за години пошуків.



Розробка | Аналітика і ТЗ | Підтримка користувачів

Чорнобай Віталій t.me/chornobai1 chornobai1@gmail.com

ПРОБЛЕМА

Конфігурація виросла. Знання про неї — ні.

2,8 млн
рядків коду у конфігурації великого холдингу

5 600+
програмних модулів

1 800+
документів і довідників — і лише кілька людей знають, як усе пов'язано

Розробник

«Де формується цей рух? Хто ще викликає процедуру? Що зламається, якщо її змінити?» — години «археології» в конфігураторі.

Аналітик

Щоб написати ТЗ, відриває розробника питанням «а як воно зараз працює?». ТЗ з білими плямами повертається на доопрацювання.

Підтримка

Заявка «не проводиться документ» ходить по колу між лініями, доки хтось не знайде причину в коді.

Бізнес

Доробки тривають довше, ніж хочеться, після релізів з'являються регресії, а ключові знання — в головах 2–3 людей.

до 58%
робочого часу розробники витрачають на розуміння вже написаного коду

Xia et al., IEEE Transactions on Software Engineering, 2018

ЧОМУ НЕ ПРОСТО CHATGPT

Загальний AI знає 1С. Він не знає вашу 1С.

	AI «з коробки»	AI + onec-ai
ЗНАННЯ	✕ Загальні знання про 1С з інтернету	✓ Ваш код, метадані, форми, ролі — актуальні щодня
ТОЧНІСТЬ	✕ Вигадує реквізити й процедури, яких у вас немає	✓ Цитує файл, процедуру й рядок. Немає факту — так і каже
ЗВ'ЯЗКИ	✕ Бачить лише фрагмент, який ви вставили в чат	✓ Граф залежностей: хто кого викликає і що зачепить зміна
ЗАЯВКИ	✕ Нічого не знає про звернення користувачів	✓ Читає заявку Service Desk / Jira разом зі скріншотами
ДОСТУП	✕ Або весь код у чат, або нічого	✓ Ролі: розробник бачить код, аналітик — логіку, користувач — метадані

РІШЕННЯ

onec-ai — шар знань між вашою 1С і AI-асистентом команди

Ваша 1С

- Конфігурація: код і метадані
- Код у довідниках (алгоритми)
- Заявки Service Desk / Jira

onec-ai

- Пошуковий індекс коду й метаданих
- Граф залежностей
- Навички й стандарти 1С
- Конектори до заявок

32 ІНСТРУМЕНТИ · MCP

AI-клієнт

- Claude (Desktop, Code)
- Cursor
- Codex

Команда

- Розробники
- Аналітики
- Підтримка й ключові користувачі

Ви обираєте AI — ми даємо йому знання

Команда працює у звичному AI-клієнті. onec-ai додає йому контекст саме вашої конфігурації.

Відкритий стандарт MCP

Model Context Protocol — відкритий протокол підключення інструментів до AI. Без прив'язки до одного вендора.

Підключення одним установником

Для кожного співробітника: без прав адміністратора і ручного редагування конфігів.

ЯК ЦЕ ПРАЦЮЄ

Від коміту в конфігурації до відповіді AI

01 · ВИВАНТАЖЕННЯ

Знімок конфігурації

Агент на Windows за розкладом вивантажує конфігурацію в Git. Робоча база при цьому не змінюється.

02 · ІНДЕКСАЦІЯ

Лише змінене

Інкрементально, за контрольними сумами файлів. Код ріжеться по процедурах, метадані — по реквізитах, з іменами й синонімами.

03 · ГРАФ

Усі зв’язки

Виклики процедур, рухи регістрів, ролі, форми, типи — і навіть код, що зберігається в даних довідників.

04 · ВІДПОВІДЬ

Гібридний пошук

Смисловий + точний + графовий пошук зливаються в один рейтинг → відповідь з посиланнями на джерело.

~10 хв
від коміту до оновленого індексу

~0,3 с
точний пошук по 2,8 млн рядків коду

32
інструменти MCP: пошук, граф, форми, ролі, заявки, збірка

84
навички для 1С: форми, ролі, СКД, БСП, запити

ДЛЯ РОЗРОБНИКІВ

Менше археології — більше розробки

- Пошук по коду й метаданих з цитатами: файл, процедура, рядок
- Аналіз впливу змін — до 5 рівнів залежностей
- Звіт ризиків по кожному релізу — автоматично після коміту
- Збірка зовнішніх обробок і звітів та перевірка XML на агенті з вашою платформою
- Стандарти коду команди — як «навички» для AI

DEV Хто викликає «ЗаполнитьПоОснованию» і що зламається, якщо змінити її параметри?

`find_callers` `impact_analysis`

DEV Яка процедура спрацьовує на кнопку «Заповнити» у формі замовлення?

`form_events` `trace_form_handler`

DEV Де формуються рухи по регістру взаєморозрахунків?

`search_code` `graph_context`

DEV Збери зовнішню обробку з цих вихідників і перевір XML

`worker: build_epf` `validate_xml`

Результат: розробник витрачає час на **рішення**, а не на пошук місця, де його застосувати.

ЯК ЦЕ ВИГЛЯДАЄ ДЛЯ РОЗРОБНИКА

Від задачі — до готового коду

AI-асистент · підключено onec-ai

Заборони проводити реалізацію, якщо клієнт у стоп-листі. Що і де змінити?

- ✓ search_code «стоп-лист клієнта» — СтопЛистСервер.КлиентЗаблокирован()
- ✓ find_callers ОбработкаПроведения — 3 виклики
- ✓ impact_analysis РеализацияТоваровУслуг — 2 звіти й обмін лише читають документ

Де: `РеализацияТоваровУслуг` → модуль об’єкта, `ОбработкаПроведения()`, рядок 42. Замініть перевірку прапорця на загальну функцію стоп-листа:

Documents/РеализацияТоваровУслуг/Ext/ObjectModule.bsl

```
41    Процедура ОбработкаПроведения(Отказ, РежимПроведения)
42  -     Если Контрагент.Заблокирован Тогда
42  +     Если СтопЛистСервер.КлиентЗаблокирован(Контрагент, Дата) Тогда
43  +         ОбщегоНазначения.СообщитьПользователю(
44  +             НСтр("ru = 'Клиент в стоп-листе'"),, "Контрагент");
45            Отказ = Истина;
46        КонецЕсли;
```

Вплив: звіти й обмін документ лише читають — зміна їх не зламає. Патч збережено як чернетку, у Конфігуратор його переносите ви.

АБО — НОВА ОБРОБКА

Зовнішня обробка за хвилини

Створи зовнішню обробку: завантаження цін постачальника з Excel

- ✓ Згенеровано форму, модуль і макет
- ✓ `worker_upload_staging` — вихідники на агенті
- ✓ `build_epf` — збірка на вашій платформі 1С
- ✓ XML перевірено, помилок немає

ЗагрузкаЦенПоставщика.epf
готово · посилання на завантаження

Ілюстративний приклад. Назви об’єктів умовні.

ДЛЯ АНАЛІТИКІВ

ТЗ на фактах, а не на припущеннях

- Структура об’єктів із синонімами — так, як бачить користувач
- Поточна логіка людською мовою — без повного доступу до вихідного коду
- Чернетки ТЗ і специфікацій в окремій зоні артефактів
- Перелік зачеплених об’єктів — точніша оцінка трудомісткості

BA Які реквізити й табличні частини має «Замовлення клієнта»? Назви — як в інтерфейсі.

`object_structure`

BA Поясни по кроках, як зараз розраховується знижка в замовленні.

`ask` `search_code`

BA Які об’єкти зачепить нове поле «Сезон» у номенклатурі?

`find_usages` `impact_analysis`

BA Склади чернетку ТЗ на доробку за заявкою ERP-1024.

`get_servicedesk_task_context` `artifact_create_draft`

Результат: менше переривань розробників, менше білих плям у ТЗ і повернень на доопрацювання.

ДЛЯ КОРИСТУВАЧІВ І ПІДТРИМКИ

Від заявки до причини — за хвилини

Користувачі звертаються як і раніше — через Service Desk або Jira. Змінюється те, що відбувається далі.



Заявка

Користувач описує проблему й додає скріншот помилки — як завжди

Номер

Фахівець передає AI номер або посилання на заявку

Контекст

AI читає опис, коментарі, погодження й вкладення

Причина

Знаходить у коді місце й умову, що викликає помилку

Рішення

Що зробити користувачу зараз — і що виправити розробнику

ЗАЗВИЧАЙ

Заявка переходить між лініями підтримки, розробник відволікається на діагностику, користувач чекає на відповідь днями.

З ONEC-AI

Перша лінія сама знаходить причину типових помилок, розробник отримує готовий діагноз із місцем у коді.

ЯК ЦЕ ВИГЛЯДАЄ

Одне питання — діагноз із місцем у коді

AI-асистент · підключено onec-ai

Заявка SD-10482: «Не проводиться реалізація — *Недостатньо товару на складі*, хоча в залишках товар є». Знайди причину.

- ✓ `get_servicedesk_task_context` — заявка, 3 коментарі, 1 скріншот
- ✓ `grep_code «Недостатньо товару»` — 4 збіги
- ✓ `find_callers ПроверитьОстатки` — 3 виклики
- ✓ `read_source ЗапасыСервер` — рядки 180–240

Причина. Під час проведення залишок контролюється в розрізі **складу й серії**. У рядках 3 і 5 серію не заповнено, тому залишок «без серії» нульовий, хоча загальний залишок товару є.

Де: `ЗапасыСервер.ПроверитьОстатки()`, рядок 214 — умова відбору за серією.

Користувачу: заповнити серію в рядках 3 і 5 і провести документ повторно.
Розробнику: пропозицію виправлення збережено як чернетку в зоні артефактів.

Конкретно, не загально

Відповідь прив’язана до вашого коду: модуль, процедура, рядок.

Два адресати

Що робити користувачу зараз — і що виправити в конфігурації.

Нічого не змінено

Виправлення — лише чернетка. Рішення приймає людина.

Ілюстративний приклад. Назви об’єктів умовні.

КЕЙС · ВЕЛИКИЙ АГРОХОЛДИНГ

Перевірено на конфігурації масштабу enterprise

Конфігурація

2,83 млн	рядків коду
5 645	модулів
1 167	документів
647	довідників

Що бачить AI

210 тис.	фрагментів у пошуковому індексі
1,97 млн	зв'язків у графі залежностей
43,9 тис.	елементів метаданих
6 200+	звичайних форм — AI читає і їх

Як працює

94%	контрольних питань — щоденний автотест якості
1,5–6 хв	щоденне оновлення індексу
0,3–3 с	типова відповідь інструменту
1 сервер	8 vCPU — для всієї команди

КЕЙС · ВИКОРИСТАННЯ

Від пілоту до щоденного інструменту за 4 місяці

41 тис.
викликів інструментів AI-асистентами з червня

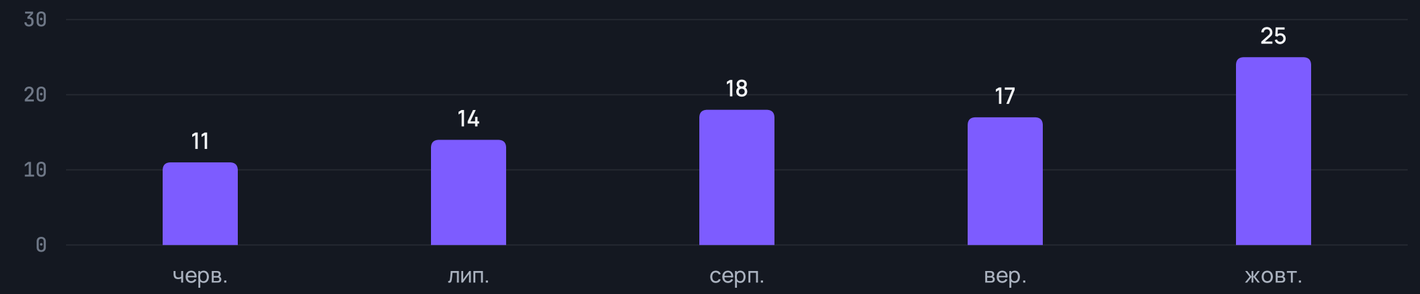
30
фахівців уже працюють із сервером

287
звернень AI до заявок Service Desk / Jira

134
підготовлені артефакти: звіти перевірок, пропозиції виправлень, ТЗ

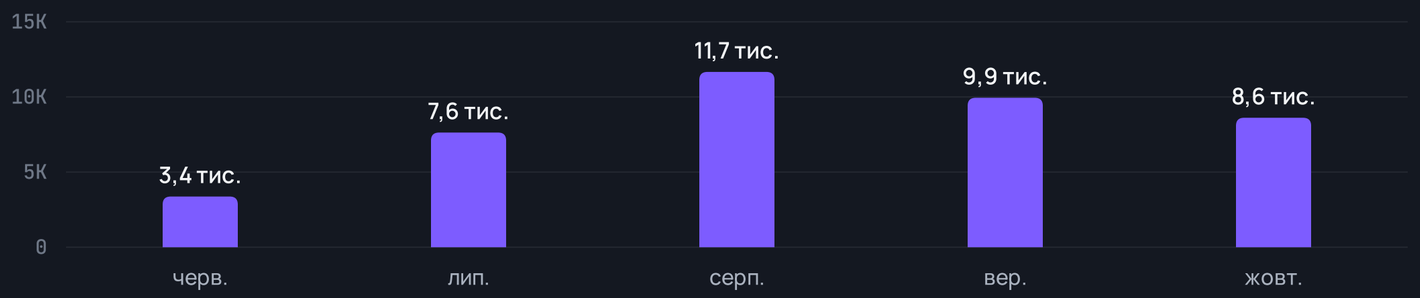
Активні користувачі на місяць

Кількість фахівців, які зверталися до сервера



Виклики інструментів на місяць

Червень — з 12.06 (запуск), жовтень — 1–21.10



ЕКОНОМІКА

Порахуйте ефект для своєї команди

Розробників 1С — 10

Аналітиків і консультантів — 4

Заявок до 2-ї/3-ї лінії на місяць — 200

Вартість години фахівця, ₴ — 600

Сценарій

Обережний | **Базовий** | Оптимістичний

ПОТЕНЦІЙНА ЕКОНОМІЯ НА РІК · ОЦІНКА

2,66 млн ₴

≈ 2,3 штатної одиниці, вивільненої під розвиток, а не під рутину

Годин на місяць	Гривень на місяць	Годин на рік
369	221 тис.	4 426

Звідки економія, годин на місяць



Розробка 192 · Аналітика 77 · Підтримка 100

Як рахуємо (сценарій «базовий»): розробник і аналітик — 160 год/міс, з яких AI знімає 12% і 12% часу на пошук і розбір коду та поточної логіки; підтримка — мінус 30 хв діагностики на заявку. Це оцінка, фактичний ефект вимірюємо в пілоті. Для порівняння: до 58% часу розробника — розуміння коду (Xia et al., 2018); задача з AI-асистентом — на 55,8% швидше (Peng et al., 2023).

ЕФЕКТ ДЛЯ БІЗНЕСУ

Більше, ніж зекономлені години

Швидше до результату

Менше часу на розбір існуючого коду — доробки потрапляють до користувачів раніше.

Менше регресій

Аналіз впливу перед зміною і автоматичний звіт ризиків після кожного релізу.

Знання не звільняються

Логіку конфігурації пояснює система, а не 2–3 незамінні людини. Нижчий ризик для бізнесу.

Швидкий онбординг

Новачок ставить питання AI, а не відволікає старших колег. Входження в конфігурацію — помітно швидше.

Розвантаження підтримки

Частину заявок перша лінія закриває сама, без ескалації до розробників.

Прозорість

Аудит запитів показує, які процеси й модулі викликають найбільше питань — де варто покращувати.

БЕЗПЕКА І КОНТРОЛЬ

AI бачить рівно те, що дозволено

Лише читання

Сервер не змінює ні конфігурацію, ні базу. Пропозиції змін — тільки чернетки в окремій зоні.

Ролі доступу

Розробник, аналітик, користувач. Повний вихідний код бачить лише роль «розробник».

Персональні ключі

У кожного співробітника свій ключ. Відкликання — в один клік, без впливу на інших.

Повний аудит

Кожен виклик фіксується: хто, який інструмент, коли і скільки тривало.

Секрети під маскою

Паролі й токени в алгоритмах із довідників маскуються ще до індексації.

Ваш контур — ваші правила

Можливе розгортання на вашому сервері. AI-клієнт і підписку обираєте ви — ми не перепродаємо моделі.

ВАРІАНТИ РОЗГОРТАННЯ

Три способи розгортання

SAAS

Хмара

Окреме ізольоване середовище для кожного клієнта: свій сервіс і своя база. Оновлення і моніторинг — на нас.

ДЛЯ КОГО

Середній бізнес, якому потрібен швидкий старт без власної інфраструктури

MANAGED ON-PREM

У вас, під нашим супроводом

Наш стек на вашому сервері. Код не залишає ваш периметр, а оновлення й підтримку беремо на себе.

ДЛЯ КОГО

Компанії, які не передають код назовні, але не хочуть утримувати DevOps під продукт

BOX

Повністю у вашому контурі

Розгортаєте й обслуговуєте самі, оновлення — офлайн-пакетами. Опційно — локальні моделі без виходу в інтернет*.

ДЛЯ КОГО

Великі компанії із закритими мережами та суворими вимогами ІБ

ЩО ПОТРІБНО ВІД ВАС

- Windows-хост з платформою 1С
- Вивантаження у форматі Конфігуратора
- Claude, Cursor або Codex

* Локальні моделі ембеддингів — у дорожній карті продукту.

ЯК СТАРТУЄМО

Пілот: ефект на ваших даних за 6–8 тижнів

ТИЖДЕНЬ 0

Демо і цілі

Показуємо систему, підписуємо NDA, домовляємося про цілі та метрики пілоту.

ТИЖДЕНЬ 1

Підключення

Агент, перше вивантаження, повна індексація, перевірка якості на ваших реальних питаннях.

ТИЖДЕНЬ 2

Команда

Ключі та установник для кожного співробітника, навчання розробників, аналітиків і підтримки.

ТИЖНІ 3–8

Робота і заміри

Щотижневий зворотний зв'язок, статистика використання, фіксація прикладів «було / стало».

Результат пілоту — звіт із виміряною економією годин, вашими прикладами і рішенням про масштабування на всю команду.